*type* specifies the way in which communication is done. In the case of Netlink, SOCK_RAW or SOCK_DGRAM can be used.

*protocol* specifies which Netlink feature is to be used. Various features are specified in *include/uapi/linux/netlink.h*, which are NETLINK_GENERIC, NETLINK_ROUTE, NETLINK_FIREWALL, etc. You can also add a custom Netlink protocol easily by adding the macro in this file.

For each Netlink protocol type, up to 32 multicast groups can be specified in code. A multicast group in Netlink is a 32-bit bitmask, where each bit represents a group. Using this multicast feature, multiple processes and kernel modules can communicate with each other with a lesser number of system calls.

To understand Netlink sockets in the user space, the following data structures need to be understood.

```
struct sockaddr_nl;
struct nlmsghdr;
struct iovec;
struct msghdr;

struct sockaddr_nl (include/uapi/linux/netlink.h)
{
__kernel_sa_family_t nl_family;
unsigned short nl_pad;
__u32 nl_pid;
__u32 nl_groups;

};
```

In the above code, let's look at what certain terms stand for.

- *nl_family:* This is the protocol family to be used, which is AF_NETLINK.
- *nl_pad*: This is used for padding.
- *nl_pid:* This is the identification or the local address of the process. It is used if a process wants to send a unicast message to other processes or the kernel.
- *nl_groups:* This is a 32-bit bitmask used for multicast communication.
- *nl_pid* can be the PID of the process, which can be initialised as follows:

```
struct sockaddr_nl addr;

addr.nl_pid = getpid();
```

If, in a process, each thread wants its own Netlink socket, then *nl_pid* can be initialised to:

```
addr.nl_pid = pthread_self() << 16 | getpid();
```

…or it can be initialised to simple numbers as:

```
addr.nl_pid = 1;
```

Or any algorithm can be used to assign it a unique value.

*nl_groups* are used for multicast communication. Each bit in this field is a multicast address. Any process which needs to listen on a particular group should set the bit.

As an example, if a process wants to listen on multicast addresses 3 and 5, then the bits are stored as follows:

```
addr.nl_groups = 1<<3 | 1<<5;
```

If, for example, a process wants to send data to multicast group 3, then it will initialise the *nl_groups* field as follows:

```
addr.nl_groups = 1 << 3;
```

If the process wants to send to both the 3 and 5 groups, then *nl_groups* will be initialised as follows:

```
addr.nl_groups = 1<<3 | 1<<5;
```

*nl_pid* is used to identify a single process or kernel and *nl_groups* is used to identify multiple processes or kernel modules, where *nl_pid* = 0 is a special address, which is the kernel address.

The kernel requires each Netlink message to include the Netlink message header. Thus a Netlink message is a combination of a message header and message payload. An application allocates a buffer long enough to store both header and payload. The starting of the buffer holds the Netlink message header and it is followed by the payload. So just by typecasting the buffer address with the header structure, the header can be accessed, after which there is the payload. The header structure (*include/uapi/linux/netlink.h*) is as follows:

```
struct nlmsghdr
{
__u32 nlmsg_len;
__u32 nlmsg_type;
__u32 nlmsg_flags;
__u32 nlmsg_seq;
__u32 nlmsg_pid;
};
```

In the code above, let's look at what certain terms mean:

- * *nlmsg_len*: This is the length of the message to be transferred, including the header length.
- * *nlmsg_type*: This is the type of message that is being transferred and is used by applications. This field is not used by the kernel.
- *nlmsg_flags*: This is used to give additional information.
- *nlmsg_seq*: This is the sequence number of the message and is used by applications. This field is not used by the kernel.
- *nlmsg_pid:* This is the identification of the process which sends the message and is used by applications. This field is not used by the kernel.